STDIN → IO	The actual standard input stream for the program. The initial value of $stdin.
STDOUT → IO	The actual standard output stream for the program. The initial value of $stdout.
SCRIPT_LINES__ → Hash	If a constant SCRIPT_LINES__ is defined and references a Hash, Ruby will store an entry containing the contents of each file it parses, with the file's name as the key and an array of strings as the value.
TOPLEVEL_BINDING → Binding	A Binding object representing the binding at Ruby's top level—the level where programs are initially executed.
TRUE → TrueClass	A reference to the object true.

The constant __FILE__ and the variable $0 can be used together to run code only if it appears in the file run directly by the user. For example, library writers often use this to include tests in their libraries that will be run if the library source is run directly, but not if the source is required into another program.

```
# library code ...

if __FILE__ == $0
  # tests...
end
```

Expressions, Conditionals, and Loops

Single terms in an expression may be any of the following:

- *Literal*. Ruby literals are numbers, strings, arrays, hashes, ranges, symbols, and regular expressions. These are described in The Basic Types, on page 433.
- *Shell command*. A shell command is a string enclosed in backquotes or in a general delimited string starting with %x. The string is executed using the host operating system's standard shell, and the resulting standard output stream is returned as the value of the expression. The execution also sets the $? variable with the command's exit status.

```
filter = "*.c"
files = `ls #{filter}`
files = %x{ls #{filter}}
```

- *Variable reference* or *constant reference*. A variable is referenced by citing its name. Depending on scope (see Scope of Constants and Variables, on page 448), you reference a constant either by citing its name or by qualifying the name, using the name of the class or module containing the constant and the scope operator (::).

```
barney    # variable reference
APP_NAMR  # constant reference
Math::PI  # qualified constant reference
```

- *Method invocation*. The various ways of invoking a method are described in Invoking a Method, on page 469.